

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-ac7e30f91e08e723c.jsonl`  
- SHA-256: `01e6971e3d01fd860be017e362017386da4036f285b25d47d3b858b1c62c697f`  
- Source modified: `2026-06-10T16:25:17+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:22:08.055Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Transient keychain read failure mints a new key and silently overwrites the only DB key
SEVERITY CLAIMED: high
FILE: src/muneem/db_key.py (line 24-31)
DESCRIPTION: `secrets.get\_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get\_or\_create\_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set\_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect\_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect\_encrypted` (src/muneem/db.py:207) calls `get\_or\_create\_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.

```
EVIDENCE QUOTED: def get_or_create_key() -> str:  
    existing = _keychain.get_secret(_DB_KEY_NAME)  
    if existing:  
        return existing  
    key = _stdlib_secrets.token_bytes(32).hex()  
    _keychain.set_secret(_DB_KEY_NAME, key)  
    return key
```

SUGGESTED FIX: Distinguish 'absent' from 'unavailable': add a get_secret variant that raises (or returns a sentinel) on KeyringError, and have get_or_create_key abort loudly instead of minting when the backend errored. Additionally, gate minting on the ledger file: if the encrypted DB file already exists but the keychain has no key, refuse with a pointer to 'muneem db recover' rather than minting a key that cannot open it.

Your job is to try to REFUTE it. Your lens: existing-mitigation: check whether something ELSE in the repo (a test, a guard, a caller-side check, a hook, config) already prevents or detects the claimed problem.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.

- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
 - Do not use WebSearch/WebFetch and do not send any repo content to any external service.
- Structured output only.

2. user (2026-06-10T16:22:14.366Z)

```
1      """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3      muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4      The 256-bit key is generated on first use and stored in the OS keychain via
5      :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6      never on disk in plaintext, never in the repo, never logged.
7
8      It is handed to SQLCipher in **raw-key form** (`PRAGMA key = "x'<64 hex>'"), which
uses
9      the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10     key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11     passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12     """
13
14     from __future__ import annotations
15
16     import secrets as _stdlib_secrets
17
18     from muneem import secrets as _keychain
19
20     # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21     _DB_KEY_NAME = "db-key"
22
23
24     def get_or_create_key() -> str:
25         """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26         existing = _keychain.get_secret(_DB_KEY_NAME)
27         if existing:
28             return existing
29         key = _stdlib_secrets.token_bytes(32).hex()
30         _keychain.set_secret(_DB_KEY_NAME, key)
31         return key
32
33
34     def peek_key() -> str | None:
35         """Return the stored key, or `None` if none has been minted yet. Never creates
one."""
36         return _keychain.get_secret(_DB_KEY_NAME)
37
38
39     def new_key() -> str:
40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47
```

3. user (2026-06-10T16:22:15.061Z)

```
1      """Secret storage via the OS keychain.
```

```

2
3     OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4     Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5     in logs. See DESIGN.md section 5.
6
7     External-call discipline (every keyring call is wrapped with attribution):
8     - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
`None`
9         (absence is a valid state; this also keeps headless CI happy).
10    - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13        """
14
15    from __future__ import annotations
16
17    import keyring
18    from keyring.errors import KeyringError, PasswordDeleteError
19
20    from muneem.errors import KeychainError
21
22    _SERVICE = "muneem"
23
24
25    def get_secret(name: str) -> str | None:
26        """Return the stored secret, or `None` if absent or the keychain is
unavailable."""
27        try:
28            return keyring.get_password(_SERVICE, name)
29        except KeyringError:
30            return None
31
32
33    def set_secret(name: str, value: str) -> None:
34        """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35        try:
36            keyring.set_password(_SERVICE, name, value)
37        except KeyringError as exc:
38            # Name only ? never the value ? in the message (it may be a password or the DB
key).
39            raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42    def delete_secret(name: str) -> None:
43        """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44        try:
45            keyring.delete_password(_SERVICE, name)
46        except (PasswordDeleteError, KeyringError):
47            pass
48

```

4. user (2026-06-10T16:22:19.748Z)

```

1     """Local SQLite store: the `documents` and `offers` slice of the schema.
2
3     This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4     (accounts, transactions, holdings, merchants, items) grows with later milestones.
5     `documents.sha256` is UNIQUE so re-ingesting the same file is a no-op.

```

```

6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      ""
8
9      from __future__ import annotations
10
11     import os
12     import sqlite3
13     from collections.abc import Iterator
14     from contextlib import contextmanager
15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25     _SCHEMA = ""
26     CREATE TABLE IF NOT EXISTS documents (
27         id            INTEGER PRIMARY KEY,
28         source        TEXT NOT NULL,
29         external_id   TEXT,
30         sender        TEXT,
31         subject       TEXT,
32         received_date TEXT,
33         doc_type      TEXT,
34         sha256        TEXT NOT NULL UNIQUE,
35         path          TEXT,
36         status        TEXT NOT NULL DEFAULT 'parsed',
37         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38     );
39
40     CREATE TABLE IF NOT EXISTS offers (
41         id            INTEGER PRIMARY KEY,
42         document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43         merchant      TEXT,
44         promo_code    TEXT,
45         description   TEXT,
46         expiry_raw    TEXT,
47         expiry_type   TEXT,
48         source        TEXT NOT NULL DEFAULT 'text',
49         redeemed      INTEGER NOT NULL DEFAULT 0,
50         source_text   TEXT
51     );
52
53     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55     -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57     -- Normalization into a unified view layer is planned for a later phase
58     -- once all bank schemas are stabilized.
59     CREATE TABLE IF NOT EXISTS axis_transactions (
60         id INTEGER PRIMARY KEY,
61         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62         account_number TEXT,
63         txn_date TEXT,
64         value_date TEXT,
65         particulars TEXT,
66         debit REAL,
67         credit REAL,
68         balance REAL

```

```

69     );
70
71     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72         id INTEGER PRIMARY KEY,
73         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74         account_number TEXT,
75         txn_date TEXT,
76         particulars TEXT,
77         merchant_category TEXT,
78         amount REAL,
79         type TEXT
80     );
81
82     CREATE TABLE IF NOT EXISTS au_transactions (
83         id INTEGER PRIMARY KEY,
84         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85         account_number TEXT,
86         txn_date TEXT,
87         value_date TEXT,
88         particulars TEXT,
89         debit REAL,
90         credit REAL,
91         balance REAL
92     );
93
94     CREATE TABLE IF NOT EXISTS hdfc_transactions (
95         id INTEGER PRIMARY KEY,
96         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97         account_number TEXT,
98         txn_date TEXT,
99         value_date TEXT,
100        particulars TEXT,
101        ref TEXT,
102        withdrawal REAL,
103        deposit REAL,
104        closing_balance REAL
105    );
106    """
107
108
109    def _raw_key_pragma(key: str) -> str:
110        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111        return f"PRAGMA key = \"x'{key}'\""
112
113
114    def is_plaintext_sqlite(db_path: Path | str) -> bool:
115        """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118        without needing the key.
119        """
120        try:
121            with open(db_path, "rb") as fh:
122                return fh.read(16) == _SQLITE_MAGIC
123        except OSError:
124            return False
125
126
127    def _harden_permissions(path: str) -> None:
128        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
`:memory:`."""
129        if path == ":memory:":
130            return
131        try:

```

```

132         os.chmod(path, 0o600)
133     except OSError:
134         pass
135
136
137     def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138         """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140         A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
141         the
142         key is wrong (or the file is not an encrypted muneem ledger), this raises.
143         """
144         try:
145             import sqlcipher3
146         except ImportError as exc: # pragma: no cover - exercised only without the binding
147             raise EncryptedDatabaseError(
148                 "database encryption requested but the 'sqlcipher3' package is not
149                 installed"
150                 ) from exc
151
152         conn = sqlcipher3.connect(path)
153         conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
154         try:
155             conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
156         except sqlcipher3.DatabaseError as exc:
157             conn.close()
158             raise EncryptedDatabaseError(
159                 f"could not open encrypted database {Path(path).name}: wrong key, or not an
160                 "
161                 "encrypted muneem ledger"
162                 ) from exc
163
164         conn.row_factory = sqlcipher3.Row
165         return conn
166
167
168     @contextmanager
169     def connect(db_path: Path | str, *, key: str | None = None) ->
170     Iterator[sqlite3.Connection]:
171         """Open a connection (commit on success, always close). Parent dirs created.
172
173         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
174         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
175         path for
176         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
177         :func:`_open_encrypted`); the production ledger is opened via
178         :func:`connect_encrypted`.
179         """
180         path = str(db_path)
181         if path != ":memory:":
182             Path(path).parent.mkdir(parents=True, exist_ok=True)
183         if key is not None:
184             conn = _open_encrypted(path, key)
185             _harden_permissions(path)
186         else:
187             conn = sqlite3.connect(path)
188             conn.row_factory = sqlite3.Row
189             conn.execute("PRAGMA foreign_keys = ON")
190         try:
191             yield conn
192             conn.commit()
193         finally:
194             conn.close()
195
196     @contextmanager

```

```

191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197         "file is not a database" error.
198         """
199         from muneem.db_key import get_or_create_key
200
201         path = str(db_path)
202         if is_plaintext_sqlite(path):
203             raise EncryptedDatabaseError(
204                 f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205                 "to migrate it to an encrypted ledger"
206             )
207         with connect(path, key=get_or_create_key()) as conn:
208             yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.
213
214         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3
219
220         path = Path(db_path)
221         if not is_plaintext_sqlite(path):
222             raise EncryptedDatabaseError(
223                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224             )
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY 'x'{key}'")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]

```

```

246         ntables = check.execute(
247             "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248         ).fetchone()[0]
249     if integrity != "ok" or ntables != src_tables:
250         raise EncryptedDatabaseError(
251             f"encryption migration verification failed for {path.name} "
252             f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253         )
254
255     backup = path.with_name(path.name + ".plaintext.bak")
256     backup.unlink(missing_ok=True)
257     path.rename(backup)
258     enc_tmp.rename(path) # consumed on success
259     _harden_permissions(str(path))
260 finally:
261     # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262     enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266 def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267     """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269     Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270     :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271     muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272     after this returns successfully.
273     """
274     conn = _open_encrypted(str(db_path), current_key) # validates current_key
275     try:
276         conn.execute(f"PRAGMA rekey = '{new_key}'")
277         conn.commit()
278     finally:
279         conn.close()
280     _harden_permissions(str(db_path))
281
282
283 def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284     """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286     Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287     naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288     it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289     """
290     import sqlcipher3
291
292     dest_path = Path(dest)
293     dest_path.parent.mkdir(parents=True, exist_ok=True)
294     dest_path.unlink(missing_ok=True)
295     source = _open_encrypted(str(db_path), key) # validates key
296     try:
297         target = sqlcipher3.connect(str(dest_path))
298     try:
299         target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300         source.backup(target) # consistent snapshot of the live DB
301     finally:
302         target.close()

```



```

303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
310         DB).
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
312         future changes
313         are forward migrations in :mod:`muneem.migrations`, recorded in
314         ``schema_migrations``.
315         """
316         from muneem import migrations
317         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
318         baseline_version=SCHEMA_VERSION)
319
320     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
321         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
322         (sha256,)).fetchone()
323
324     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
325         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
326         txns."""
327         cols = ", ".join(fields)
328         placeholders = ", ".join("?" for _ in fields)
329         cur = conn.execute(
330             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
331             tuple(fields.values()),
332         )
333         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
334         checker
335         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
336             raise RuntimeError(f"INSERT into {table} did not return a row id")
337         return int(rowid)
338
339     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
340         """Insert into documents table. Delegates to the shared helper for DRY."""
341         return _insert_row(conn, "documents", dict(fields))
342
343     def record_provenance(
344         conn: sqlite3.Connection,
345         *,
346         document_id: int,
347         source_type: str,
348         parser: str | None = None,
349         parser_version: str | None = None,
350         mapping_layer: str | None = None,
351         confidence: str | None = None,
352     ) -> None:
353         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)`, so
355         a single
356         provenance record per document is reachable by join. Idempotent on ``document_id``
357         (a re-parse
358         overwrites rather than duplicating).
359         """
360         conn.execute(

```

```

359         "INSERT INTO provenance "
360         "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361         "VALUES (?, ?, ?, ?, ?, ?) "
362         "ON CONFLICT(document_id) DO UPDATE SET "
363         "source_type=excluded.source_type, parser=excluded.parser, "
364         "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365         "confidence=excluded.confidence",
366         (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367     )
368
369
370     def record_parse(
371         conn: sqlite3.Connection,
372         document_id: int,
373         *,
374         confidence: str,
375         parser: str,
376         source_type: str = "pdf_statement",
377         parser_version: str | None = None,
378         mapping_layer: str = "deterministic",
379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
381         parse)."""
382         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
383         document_id))
384         record_provenance(
385             conn,
386             document_id=document_id,
387             source_type=source_type,
388             parser=parser,
389             parser_version=parser_version,
390             mapping_layer=mapping_layer,
391             confidence=confidence,
392         )
393
394     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
395         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
396         (document_id,)).fetchone()
397
398
399     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
400         return _insert_row(conn, "offers", fields)
401
402
403     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
404         return conn.execute(
405             """
406             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
407             d.external_id, d.path
408             FROM offers o
409             JOIN documents d ON d.id = o.document_id
410             ORDER BY o.id
411             """
412         ).fetchall()
413
414
415     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
416         return _insert_row(conn, "axis_transactions", fields)
417
418
419     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
420         return _insert_row(conn, "axis_cc_transactions", fields)

```

```

421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

5. user (2026-06-10T16:22:21.035Z)

Found 23 files

- src\muneem\db.py
- DESIGN.md
- docs\feedbackJune2026.md
- docs\ROADMAP.md
- docs\CURRENT_STATE.md
- tests\test_recovery.py
- tests\test_precommit_hook.py
- src\muneem\recovery.py
- tests\test_dependency_licenses.py
- src\muneem\cli.py
- docs\DEPENDENCIES.md
- docs\encryption-at-rest.md
- .github\precommit_check.py
- src\muneem\db_key.py
- tests\test_db_crypto_ops.py
- tests\test_db_crypto.py
- tests\parsers\test_savings.py
- src\muneem\parsers\savings.py
- tests\test_ocr.py
- docs\prior-art-email-mining.md
- tests\test_promo_regression.py
- docs\TESTING.md
- testdata\promo-emails\README.md

6. user (2026-06-10T16:22:30.425Z)

```

1      """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.
2
3      The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
? OS
4      reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
same-key
5      backup** become permanently unrecoverable. This module lets the user wrap that key under
a
6      passphrase they choose, into a recovery file they back up **off-machine**; with the file
*and* the
7      passphrase
8      they can restore the key into a fresh keychain.
9
10     Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
passphrase + a random
11     salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
12     ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
passphrase. A wrong
13     passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
14     """
15
16     from __future__ import annotations
17
18     import base64
19     import json
20     import os
21     from pathlib import Path
22     from typing import Any
23
24     from cryptography.exceptions import InvalidTag
25     from cryptography.hazmat.primitives.ciphers.aead import AESGCM
26     from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
27
28     from muneem.errors import MuneemError
29
30     RECOVERY_VERSION = 1
31     _AAD = b"muneem-recovery-v1" # binds the ciphertext to this format/version
32
33     # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
they can be
34     # raised over time without breaking older recovery files.
35     _TIME_COST = 3
36     _MEMORY_KIB = 64 * 1024 # 64 MiB
37     _LANES = 4
38     _SALT_BYTES = 16
39     _NONCE_BYTES = 12
40     _KEY_BYTES = 32
41     _MIN_PASSPHRASE = 8
42
43
44     class RecoveryError(MuneemError):
45         """A recovery file could not be created, or could not be opened (wrong passphrase /
corrupt)."""
46
47
48     def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
int) -> bytes:
49         kdf = Argon2id(
50             salt=salt,
51             length=_KEY_BYTES,
52             iterations=time_cost,
53             lanes=lanes,
54             memory_cost=memory_kib,
55         )
56         return kdf.derive(passphrase.encode("utf-8"))

```

```

57
58
59     def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
60         """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
61         ``passphrase``."""
62         if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
63             raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
characters")
64         salt = os.urandom(_SALT_BYTES)
65         nonce = os.urandom(_NONCE_BYTES)
66         key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
lanes=_LANES)
67         ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
68         return {
69             "version": RECOVERY_VERSION,
70             "kdf": {
71                 "algo": "argon2id",
72                 "time_cost": _TIME_COST,
73                 "memory_kib": _MEMORY_KIB,
74                 "lanes": _LANES,
75             },
76             "salt": base64.b64encode(salt).decode(),
77             "nonce": base64.b64encode(nonce).decode(),
78             "ciphertext": base64.b64encode(ciphertext).decode(),
79         }
80
81     def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
82         """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
RecoveryError."""
83         try:
84             salt = base64.b64decode(payload["salt"])
85             nonce = base64.b64decode(payload["nonce"])
86             ciphertext = base64.b64decode(payload["ciphertext"])
87             kdf = payload.get("kdf", {})
88         except (KeyError, ValueError, TypeError) as exc:
89             raise RecoveryError("recovery file is malformed") from exc
90         key = _derive(
91             passphrase,
92             salt,
93             time_cost=int(kdf.get("time_cost", _TIME_COST)),
94             memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
95             lanes=int(kdf.get("lanes", _LANES)),
96         )
97         try:
98             plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
99         except InvalidTag as exc:
100             raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
101         return plain.decode("utf-8")
102
103
104     def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
105         """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only
perms)."""
106         payload = wrap_key(db_key, passphrase)
107         out = Path(path)
108         out.parent.mkdir(parents=True, exist_ok=True)
109         out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
110         try:
111             os.chmod(out, 0o600)
112         except OSError: # pragma: no cover - best-effort on platforms without POSIX perms
113             pass
114         return out
115
116

```

```

117     def read_recovery_file(path: Path | str, passphrase: str) -> str:
118         """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
failure."""
119         p = Path(path)
120         try:
121             payload = json.loads(p.read_text(encoding="utf-8"))
122             except (OSError, ValueError) as exc:
123                 raise RecoveryError(f"could not read recovery file {p.name}") from exc
124         return unwrap_key(payload, passphrase)
125

```

7. user (2026-06-10T16:22:30.885Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30     app.add_typer(offers_app, name="offers")
31
32     txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33     app.add_typer(txns_app, name="txns")
34
35     db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36     app.add_typer(db_app, name="db")
37
38
39     @contextmanager
40     def _ledger(db_path: Path):
41         """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
42         error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43         try:
44             with dbmod.connect_encrypted(db_path) as conn:
45                 yield conn
46         except dbmod.EncryptedDatabaseError as exc:
47             typer.echo(str(exc), err=True)
48             raise typer.Exit(1) from exc
49
50
51     # Dispatch table for supported banks (extensible as new parsers land).

```

```

52     BANK_DISPATCH = {
53         "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54         "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55         "au": (dbmod.list_au_transactions, "debit", "credit"),
56         "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57     }
58
59
60     def _version_callback(value: bool) -> None:
61         if value:
62             typer.echo(f"muneem {__version__}")
63             raise typer.Exit()
64
65
66     @app.callback()
67     def main(
68         version: bool = typer.Option(
69             False,
70             "--version",
71             callback=_version_callback,
72             is_eager=True,
73             help="Show the muneem version and exit.",
74         ),
75     ) -> None:
76         """muneem command-line interface."""
77
78
79     @app.command()
80     def info() -> None:
81         """Show resolved local paths (no secrets, no sensitive data)."""
82         cfg = load_config()
83         present = "present" if cfg.config_file.is_file() else "absent"
84         typer.echo(f"muneem {__version__}")
85         typer.echo(f"config dir : {cfg.config_dir}")
86         typer.echo(f"data dir   : {cfg.data_dir}")
87         typer.echo(f"db path   : {cfg.db_path}")
88         typer.echo(f"staging  : {cfg.staging_dir}")
89         typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92     def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
93 bool) -> None:
94         """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
95
96         The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
97 extraction,
98 since the text needed to pick a parser is itself encrypted. The resolved password is
99 forwarded
100 to extraction and the parser, used in-memory only ? never logged.
101 """
102 from muneem import db as dbmod
103 from muneem.extract import extract
104 from muneem.parsers import get_parser
105 from muneem.statement_unlock import resolve_password
106 from muneem.unlock import is_encrypted
107
108 password: str | None = ""
109 if is_encrypted(pdf):
110     password = resolve_password(pdf, interactive=interactive)
111     if password is None:
112         hint = " (run without --no-prompt to enter it)" if not interactive else ""
113         typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
114 err=True)
115         raise typer.Exit(1)

```

```

113     ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
114     str
115     parser = get_parser(ex.text)
116     if parser is None:
117         typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
118 err=True)
119         raise typer.Exit(1)
120
121     with _ledger(db_path) as conn:
122         dbmod.init_db(conn)
123         existing = dbmod.get_document_by_sha(conn, ex.sha256)
124         if existing is not None:
125             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
126             return
127         doc_id = dbmod.insert_document(
128             conn,
129             source="file",
130             external_id=pdf.name,
131             doc_type=doc_type,
132             sha256=ex.sha256,
133             path=str(pdf.resolve()),
134             status="parsed",
135         )
136         count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
137         doc = dbmod.get_document_by_sha(conn, ex.sha256)
138         status = doc["status"] if doc else "parsed"
139
140     typer.echo(
141         f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
142         f"via {parser.__class__.__name__} [{status}]"
143     )
144
145 def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
146     """Run one statement through the full parse pipeline into an in-memory DB.
147
148     Returns `(outcome, row_count, parser_name)` where `outcome` is the document
149     status
150     (`reconciled` / `unreconciled`) or a failure tag (`locked` / `no-parser` /
151     `error`).
152     Nothing is persisted and no statement value is surfaced ? only the verdict and
153     counts.
154     """
155     from muneem import db as dbmod
156     from muneem.extract import extract
157     from muneem.parsers import get_parser
158     from muneem.statement_unlock import resolve_password
159     from muneem.unlock import is_encrypted
160
161     password = ""
162     if is_encrypted(pdf):
163         password = resolve_password(pdf, interactive=interactive) or ""
164         if not password:
165             return ("locked", 0, None)
166     try:
167         ex = extract(pdf, password=password)
168         parser = get_parser(ex.text)
169     except Exception as exc: # eval must not crash on one bad file; record the redacted
170         reason
171         _log.warning(
172             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
173             type(exc).__name__
174         )
175         return ("error", 0, None)
176     if parser is None:

```



```

171         return ("no-parser", 0, None)
172     name = parser.__class__.__name__
173     try:
174         with dbmod.connect(":memory:") as conn:
175             dbmod.init_db(conn)
176             doc_id = dbmod.insert_document(
177                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178             )
179             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180             doc = dbmod.get_document_by_sha(conn, ex.sha256)
181             status = doc["status"] if doc else "parsed"
182     except Exception as exc:
183         _log.warning(
184             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185         )
186         return ("error", 0, name)
187     return (status, count, name)
188
189
190 @app.command("eval")
191 def eval_statements(
192     folder: Path = typer.Argument(
193         ...,
194         exists=True,
195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",
202         help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203     ),
204 ) -> None:
205     """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207     The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208     statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209     an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210     """
211     pdfs = sorted(folder.glob("*.pdf"))
212     if not pdfs:
213         typer.echo(f"no .pdf files in {folder}")
214         return
215     reconciled = 0
216     for pdf in pdfs:
217         outcome, count, name = _eval_statement(pdf, interactive=prompt)
218         reconciled += outcome == "reconciled"
219         typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220     typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223 @app.command()
224 def ingest(
225     pdf: Path = typer.Argument(
226         ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
227     ),

```

```

228     doc_type: str = typer.Option("promo", help="Document category."),
229     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230     ocr: bool = typer.Option(
231         False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232     ),
233     no_prompt: bool = typer.Option(
234         False,
235         "--no-prompt",
236         help="Never prompt for a statement password; use only cached/configured
candidates.",
237     ),
238 ) -> None:
239     """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240     from muneem.extract import extract
241     from muneem.offers import extract_offer
242
243     db_path = db or load_config().db_path
244
245     if doc_type == "bank_statement":
246         _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247         return
248
249     from muneem import db as dbmod
250
251     ex = extract(pdf)
252     text_fields = extract_offer(ex.text)
253     # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254     found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255     expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257     if ocr:
258         from muneem.ocr import ocr_pdf, tesseract_available
259
260         if not tesseract_available():
261             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262         else:
263             ocr_fields = extract_offer(ocr_pdf(pdf))
264             known = {c for c, _ in found}
265             for code in ocr_fields.codes:
266                 if code not in known:
267                     found.append((code, "ocr"))
268                 known.add(code)
269             if expiry_raw is None:
270                 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(
289                 conn,

```

```

290         document_id=doc_id,
291         promo_code=code,
292         expiry_raw=expiry_raw,
293         expiry_type=expiry_type,
294         source=src,
295         source_text=ex.text[:2000],
296     )
297
298     summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
305 dir)."),
306 ) -> None:
307     """List extracted offers."""
308     from muneem import db as dbmod
309
310     db_path = db or load_config().db_path
311     if not Path(db_path).exists():
312         typer.echo("no offers yet (database not created).")
313         return
314
315     with _ledger(db_path) as conn:
316         dbmod.init_db(conn)
317         rows = dbmod.list_offers(conn)
318
319     if not rows:
320         typer.echo("no offers found.")
321         return
322
323     for row in rows:
324         who = row["merchant"] or row["external_id"] or "?"
325         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
326         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
327         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
328
329
330 @txns_app.command("list")
331 def txns_list(
332     bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
333 axis)."),
334     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
335 dir)."),
336     limit: int = typer.Option(50, help="Max number of transactions to show."),
337 ) -> None:
338     """List parsed transactions for a specific bank."""
339     from muneem import db as dbmod
340
341     db_path = db or load_config().db_path
342     if not Path(db_path).exists():
343         typer.echo("database not created yet.")
344         return
345
346     bank = bank.lower()
347     if bank not in BANK_DISPATCH:
348         typer.echo(f"Unknown bank: {bank}", err=True)
349         return
350
351     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
352     with _ledger(db_path) as conn:
353         dbmod.init_db(conn)
354         rows = list_fn(conn, limit=limit)

```

```

352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted).")
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."

```

```

410         )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
415         used."""
416         from muneem import db as dbmod
417         from muneem.db_key import peek_key
418
419         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
420             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
421             err=True)
422             raise typer.Exit(1)
423         key = peek_key()
424         if not key:
425             typer.echo("no encryption key in the keychain for this ledger.", err=True)
426             raise typer.Exit(1)
427         return key
428
429     @db_app.command("rotate-key")
430     def db_rotate_key(
431         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
432         dir)."),
433     ) -> None:
434         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
435         from muneem import db as dbmod
436         from muneem import db_key
437
438         db_path = db or load_config().db_path
439         current = _require_encrypted_ledger(db_path)
440         fresh = db_key.new_key()
441         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
442         the file.
443         db_key.set_key(fresh)
444         try:
445             dbmod.rekey_database(db_path, current, fresh)
446         except dbmod.EncryptedDatabaseError as exc:
447             db_key.set_key(current) # roll back; the file is still under the old key
448             typer.echo(str(exc), err=True)
449             raise typer.Exit(1) from exc
450         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
451
452     @db_app.command("backup")
453     def db_backup(
454         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
455         copy."),
456         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
457         dir)."),
458     ) -> None:
459         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
460         active)."""
461         from muneem import db as dbmod
462
463         db_path = db or load_config().db_path
464         key = _require_encrypted_ledger(db_path)
465         try:
466             dbmod.backup_database(db_path, dest, key)
467         except dbmod.EncryptedDatabaseError as exc:
468             typer.echo(str(exc), err=True)
469             raise typer.Exit(1) from exc
470         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")

```

```

468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json).",
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
477         lost. The
478         file holds only the *wrapped* key ? useless without the passphrase.
479         """
480         import getpass
481
482         from muneem import recovery
483         from muneem.db_key import peek_key
484
485         key = peek_key()
486         if not key:
487             typer.echo("no encryption key in the keychain yet (create the ledger first).",
488 err=True)
489             raise typer.Exit(1)
490         out_path = out or (load_config().data_dir / "muneem-recovery.json")
491         passphrase = getpass.getpass("New recovery passphrase: ")
492         if passphrase != getpass.getpass("Confirm passphrase: "):
493             typer.echo("passphrases did not match.", err=True)
494             raise typer.Exit(1)
495         try:
496             recovery.write_recovery_file(out_path, key, passphrase)
497         except recovery.RecoveryError as exc:
498             typer.echo(str(exc), err=True)
499             raise typer.Exit(1) from exc
500         typer.echo(
501             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
502 / "
503         "separate drive) and remember the passphrase ? you need BOTH to recover. Without
504 them, a "
505         "lost keychain means the ledger and its backups are unrecoverable."
506     )
507
508     @db_app.command("recover")
509     def db_recover(
510         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
511 path."),
512         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
513 dir)."),
514     ) -> None:
515         """Restore the ledger's encryption key from a recovery file + passphrase (into the
516 keychain)."""
517         import getpass
518
519         from muneem import db as dbmod
520         from muneem import db_key, recovery
521
522         passphrase = getpass.getpass("Recovery passphrase: ")
523         try:
524             key = recovery.read_recovery_file(file, passphrase)
525         except recovery.RecoveryError as exc:
526             typer.echo(str(exc), err=True)
527             raise typer.Exit(1) from exc
528         db_path = db or load_config().db_path
529         # If the ledger is present, verify the recovered key actually opens it before
530 committing it.
531         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):

```

```

525         try:
526             with dbmod.connect(db_path, key=key) as conn:
527                 conn.execute("SELECT 1")
528         except dbmod.EncryptedDatabaseError as exc:
529             typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530             raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

8. user (2026-06-10T16:22:54.473Z)

docs\CURRENT_STATE.md:35:[Omitted long matching line]

docs\CURRENT_STATE.md:41:* **The on-disk ledger is encrypted at rest** (decision D.2) ?
SQLCipher transparent AES-256 via `sqlcipher3`, opened through `db.connect\_encrypted()`; the 256-bit key lives in the OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext stdlib `sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).

docs\CURRENT_STATE.md:43:* Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement passwords, **the DB encryption key**) via keyring.

docs\CURRENT_STATE.md:50:* Local-only tests for real statements (gitignored data; passwords via env/keyring, never committed). The suite never touches the real OS keychain (autouse fixture in `tests/conftest.py`) ? headless-CI safe.

docs\CURRENT_STATE.md:53:Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis 6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:

docs\CURRENT_STATE.md:62:The project stays **local-first**, secrets in the OS keychain, promo corpus safe for CI/LLM experiments.

docs\feedbackJune2026.md:38:| "key rotation scaffolding (the deferred passphrase path)" |
Rotation done in #39. `rekey\_database()` + `muneem db rotate-key` (keychain-first with rollback). | The **passphrase path** specifically ? but reframed as **recovery**, not just "stronger" (see §2). |

docs\feedbackJune2026.md:52:> The 256-bit DB key lives only in the OS keychain. **`muneem db backup` encrypts the backup with the

docs\feedbackJune2026.md:53:> same key.** So if the keychain is lost ? OS reinstall, profile/Credential-Manager corruption, dead

docs\feedbackJune2026.md:63: manager / printed). Losing the keychain ? restore the key from passphrase + recovery file.

docs\feedbackJune2026.md:67: backup is restorable even with the keychain gone.

docs\feedbackJune2026.md:68:- Ship a **`docs/recovery.md` runbook**: "keychain lost," "move to a new machine," "verify a backup."

docs\feedbackJune2026.md:160:is P0 ? pull it into Phase A, not "later."** Document backup/restore + "keychain lost." **Phase A

docs\feedbackJune2026.md:168: Keychain) and sqlcipher3 wheels are per-OS. This directly de-risks "works on my machine" for the

docs\encryption-at-rest.md:29: **OS keychain** via `muneem.secrets` (`muneem`/`db-key`) ? Windows Credential Manager (DPAPI),

docs\encryption-at-rest.md:30: macOS Keychain, or Secret Service on Linux. Same mechanism as OAuth tokens.

docs\encryption-at-rest.md:38:file (`PRAGMA rekey`), and updates the keychain ? keychain-first with rollback, so a failed file

docs\encryption-at-rest.md:39:rekey leaves the keychain matching the file. Rotate on suspected compromise; no scheduled rotation

docs\encryption-at-rest.md:44:Because the key lives only in the keychain and `db backup` encrypts backups with that *same* key, a

docs\encryption-at-rest.md:45:**lost keychain** (OS reinstall, new machine, profile corruption) would otherwise destroy the ledger

docs\encryption-at-rest.md:55: keychain. You need **both** the file and the passphrase.

docs\encryption-at-rest.md:58:derived from a passphrase, never stored) for stronger-than-keychain protection on a shared machine ?

docs\encryption-at-rest.md:59:distinct from the recovery file above, which keeps the frictionless keychain key as primary.

docs\encryption-at-rest.md:63:Keychain + SQLCipher ****defends against****: a stolen or discarded ****disk/file****, the DB ending up in

docs\encryption-at-rest.md:65:****malware running as the logged-in user**** ? that attacker can read the keychain and dump decrypted

docs\encryption-at-rest.md:85:(`tests/test\_db\_crypto.py`). The test suite never touches the real keychain (autouse fixture in

docs\TESTING.md:14:4. ****CI is offline, deterministic, secretless.**** No network, no cloud LLM, no OAuth, no keychain in the default CI run. Non-deterministic steps (OCR, LLM) are mocked/cassette-backed by default and exercised live only behind explicit markers.

docs\TESTING.md:167:- ****Token storage.**** Token persistence goes through the `keyring` abstraction (mocked in tests); a test asserts nothing is written to a plaintext file on disk.

docs\DEPENDENCIES.md:28:| keyring | MIT | OS keychain (secrets) |

docs\ROADMAP.md:13:> ****Progress snapshot (2026-06):**** Milestone 0 (scaffolding / offline CI / config + keyring) ? and

docs\ROADMAP.md:18:> (C.1 ? hybrid candidates ? prompt, keychain-cached; see ingesting-statements.md). ****Not yet:**** the

docs\ROADMAP.md:71:****Still parked**** (do not pick silently ? see § 9 below): § 8.3 LLM model for ***sensitive*** categories, and § 8.5 merchant canonicalization. **_(Encryption-at-rest ? D.2 ? is now ****decided & implemented****: SQLCipher via `sqlcipher3` + keychain key. See [docs/encryption-at-rest.md](./encryption-at-rest.md))._**

docs\ROADMAP.md:80:4. ****Secrets discipline.**** OAuth tokens ? OS keychain (Windows Credential Manager). Nothing sensitive in git; don't weaken `.gitignore`. Logs are redacted (no account numbers, balances, line items, or statement text).

docs\ROADMAP.md:100:| Secret storage | ****keyring**** | Uses Windows Credential Manager on win32. |

docs\ROADMAP.md:101:| DB | ****SQLCipher via `sqlcipher3`**** (stdlib `sqlite3` API) + lightweight versioned migrations | ****Encryption-at-rest implemented**** (transparent AES-256; 256-bit key in the OS keychain). **`:memory:`** tests stay on plaintext stdlib `sqlite3`. See [docs/encryption-at-rest.md](./encryption-at-rest.md). |

docs\ROADMAP.md:143:- [x] ****0.3 (S)**** Config + secrets foundation: a config loader (env + a gitignored `config.toml`), and a `keyring`-backed secret accessor stub. Confirm `.gitignore` covers `config.toml` and any new secret paths.

docs\ROADMAP.md:161:- [] ****B.1 (M)**** Gmail OAuth2: register a Google Cloud project, `gmail.readonly` scope, store the token in `keyring`. Document the one-time consent flow.

docs\ROADMAP.md:171:- [x] ****C.1 (M)**** Unlocking (Stage 2), the ****hybrid**** strategy ? ****wired into `muneem ingest`**** (`statement\_unlock.py`): keychain-cached + `password\_rules.yaml` candidates ? interactive prompt, with the winning password cached in the keychain. ****No password material in repo or logs.**** Decrypt via pypdf/pdfminer/pdfplumber (pikepdf was the flagged fallback, not used). See [ingesting-statements.md](./ingesting-statements.md).

docs\ROADMAP.md:179:- [x] ****D.2 (S)**** Encryption-at-rest decision (DESIGN § 5.4): ****DONE (2026-05)**** ? adopted SQLCipher now (transparent AES-256 via `sqlcipher3`), 256-bit key in the OS keychain; queryable, permissive, Windows+Linux wheels. Research-backed (see § 9 forced-decision note). Details in [docs/encryption-at-rest.md](./encryption-at-rest.md).

docs\ROADMAP.md:200:- ****Security & secrets hygiene**** ? keychain for tokens + cached passwords; redacted logs; gitignore audits each milestone; encryption-at-rest tracked to D.2.

docs\ROADMAP.md:297:- ****~****Encryption-at-rest timing**** (D.2) ? SQLCipher now vs. later.~****

****Resolved 2026-05:**** SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing real numbers; research-backed. See [docs/encryption-at-rest.md](./encryption-at-rest.md).

docs\onboarding-a-bank.md:25:muneem eval testdata/<bank>-statements/ # uses password_rules.yaml / keychain to unlock

docs\onboarding-a-bank.md:54:- [] Real seeds in a gitignored folder; password in `password\_rules.yaml` (config dir) or keychain.

docs\ingesting-statements.md:15:1. ****Cached candidates**** ? passwords from earlier successful unlocks, stored in your ****OS keychain****.

docs\ingesting-statements.md:18: ****cached in the keychain****, so the next ingest of any statement with the same password is

docs\ingesting-statements.md:76: live only in memory and the ****OS keychain**** (Windows Credential Manager / macOS Keychain / Linux

docs\ingesting-statements.md:93:Keychain storage needs a backend. Use a desktop keyring (GNOME Keyring / KWallet), or

docs\ingesting-statements.md:94:`pip install keyrings.alt` for a file-based backend. Until then, `password\_rules.yaml` + `--no-prompt`

docs\ingesting-statements.md:95:works without any keychain (you just won't get prompt-caching).

9. user (2026-06-10T16:22:55.602Z)

```
tests\test_db_crypto.py:94:def test_get_or_create_key_is_idempotent_and_64_hex(monkeypatch):
tests\test_db_crypto.py:99:     k1 = db_key.get_or_create_key()
tests\test_db_crypto.py:101:     k2 = db_key.get_or_create_key()
tests\test_db_invariants.py:231:     """If the SQLCipher binding is unavailable, a keyed open
must RAISE ? never quietly write a
tests\parsers\test_validation.py:87:def
test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
tests\test_secrets.py:9:from keyring.errors import KeyringError
tests\test_secrets.py:20:def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):
tests\test_secrets.py:22:     raise KeyringError("no backend")
tests\test_secrets.py:36:     raise KeyringError("locked")
tests\test_secrets.py:48:     raise KeyringError("nope")
```

10. assistant (2026-06-10T16:23:02.352Z)

The swallow behavior is deliberate and tested. Now let me check the installed keyring Windows backend to assess how realistic "read raises KeyringError, then write succeeds" is.

11. user (2026-06-10T16:23:04.524Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import keyring.backends.Windows as w; print(w.__file__)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'keyring'
```

12. user (2026-06-10T16:23:04.530Z)

```
1     """The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed
delete.
2
3     The autouse confstest fixture swaps ``muneem.secrets`` for an in-memory store, so we
capture the
4     *real* wrapper functions at import time and drive them by patching the underlying
``keyring`` lib.
5     """
6
7     import keyring
8     import pytest
9     from keyring.errors import KeyringError
10
11     import muneem.secrets as secretsmod
12     from muneem.errors import KeychainError
13
14     # Captured before the autouse keychain fixture rebinds the module attributes per-test.
15     _real_get = secretsmod.get_secret
16     _real_set = secretsmod.set_secret
17     _real_delete = secretsmod.delete_secret
18
19
20     def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):
21         def boom(service, name):
22             raise KeyringError("no backend")
23
24         monkeypatch.setattr(keyring, "get_password", boom)
25         assert _real_get("anything") is None # graceful: absence, not a crash
26
27
28     def test_get_secret_returns_the_value(monkeypatch):
29         monkeypatch.setattr(keyring, "get_password", lambda s, n: "v" if n == "k" else None)
30         assert _real_get("k") == "v"
```

```

31         assert _real_get("missing") is None
32
33
34     def test_set_secret_raises_keychain_error_with_name_but_not_value(monkeypatch):
35         def boom(service, name, value):
36             raise KeyringError("locked")
37
38         monkeypatch.setattr(keyring, "set_password", boom)
39         with pytest.raises(KeychainError) as ei:
40             _real_set("db-key", "super-secret-value")
41         msg = str(ei.value)
42         assert "db-key" in msg # the name is safe to surface
43         assert "super-secret-value" not in msg # the value must NEVER appear
44
45
46     def test_delete_secret_swallows_keyring_errors(monkeypatch):
47         def boom(service, name):
48             raise KeyringError("nope")
49
50         monkeypatch.setattr(keyring, "delete_password", boom)
51         _real_delete("k") # must not raise
52

```

13. user (2026-06-10T16:23:12.135Z)

Exit code 1

```

.claude
.githubhooks
.github
.mypy_cache
.pytest_cache
.ruff_cache
.venv
docs
src
testdata
tests
tools
.coverage
.gitattributes
.gitignore
CLAUDE.md
DESIGN.md
Makefile
pyproject.toml
README.md

```

Traceback (most recent call last):

```

  File "<string>", line 1, in <module>
    import keyring.backends.Windows as w; print(w.__file__); import keyring;
print(keyring.__version__)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'keyring' has no attribute '__version__'
C:\Users\avidu\Projects\muneeem\.venv\Lib\site-packages\keyring\backends\Windows.py

```

14. user (2026-06-10T16:23:16.265Z)

```

1     from __future__ import annotations
2
3     import logging
4
5     from jaraco.context import ExceptionTrap
6
7     from ..backend import KeyringBackend
8     from ..compat import properties
9     from ..credentials import SimpleCredential

```

```

10 from ..errors import PasswordDeleteError
11
12 with ExceptionTrap() as missing_deps:
13     try:
14         # prefer pywin32-ctypes
15         from win32ctypes.pywin32 import pywintypes, win32cred
16
17         # force demand import to raise ImportError
18         win32cred.__name__ # noqa: B018
19     except ImportError:
20         # fallback to pywin32
21         import pywintypes
22         import win32cred
23
24         # force demand import to raise ImportError
25         win32cred.__name__ # noqa: B018
26
27 log = logging.getLogger(__name__)
28
29
30 class Persistence:
31     def __get__(self, keyring, type=None):
32         return getattr(keyring, '_persist', win32cred.CRED_PERSIST_ENTERPRISE)
33
34     def __set__(self, keyring, value):
35         """
36         Set the persistence value on the Keyring. Value may be
37         one of the win32cred.CRED_PERSIST_* constants or a
38         string representing one of those constants. For example,
39         'local machine' or 'session'.
40         """
41         if isinstance(value, str):
42             attr = 'CRED_PERSIST_' + value.replace(' ', '_').upper()
43             value = getattr(win32cred, attr)
44             keyring._persist = value
45
46
47 class DecodingCredential(dict):
48     @property
49     def value(self):
50         """
51         Attempt to decode the credential blob as UTF-16 then UTF-8.
52         """
53         cred = self['CredentialBlob']
54         try:
55             return cred.decode('utf-16')
56         except UnicodeDecodeError:
57             decoded_cred_utf8 = cred.decode('utf-8')
58             log.warning(
59                 "Retrieved a UTF-8 encoded credential. Please be aware that "
60                 "this library only writes credentials in UTF-16."
61             )
62             return decoded_cred_utf8
63
64
65 class WinVaultKeyring(KeyringBackend):
66     """
67     WinVaultKeyring stores encrypted passwords using the Windows Credential
68     Manager.
69
70     Requires pywin32
71
72     This backend does some gymnastics to simulate multi-user support,
73     which WinVault doesn't support natively. See
74     https://github.com/jaraco/keyring/issues/47#issuecomment-75763152

```

```

75     for details on the implementation, but here's the gist:
76
77     Passwords are stored under the service name unless there is a collision
78     (another password with the same service name but different user name),
79     in which case the previous password is moved into a compound name:
80     {username}@{service}
81     """
82
83     persist = Persistence()
84
85     @properties.classproperty
86     def priority(cls) -> float:
87         """
88         If available, the preferred backend on Windows.
89         """
90         if missing_deps:
91             raise RuntimeError("Requires Windows and pywin32")
92         return 5
93
94     @staticmethod
95     def _compound_name(username, service):
96         return f'{username}@{service}'
97
98     def get_password(self, service, username):
99         res = self._resolve_credential(service, username)
100         return res and res.value
101
102     def _resolve_credential(
103         self, service: str, username: str | None
104     ) -> DecodingCredential | None:
105         # first attempt to get the password under the service name
106         res = self._read_credential(service)
107         if not res or username and res['UserName'] != username:
108             # It wasn't found so attempt to get it with the compound name
109             res = self._read_credential(self._compound_name(username, service))
110         return res
111
112     def _read_credential(self, target):
113         try:
114             res = win32cred.CredRead(
115                 Type=win32cred.CRED_TYPE_GENERIC, TargetName=target
116             )
117         except pywintypes.error as e:
118             if e.winerror == 1168 and e.funcname == 'CredRead': # not found
119                 return None
120             raise
121         return DecodingCredential(res)
122
123     def set_password(self, service, username, password):
124         existing_pw = self._read_credential(service)
125         if existing_pw:
126             # resave the existing password using a compound target
127             existing_username = existing_pw['UserName']
128             target = self._compound_name(existing_username, service)
129             self._set_password(
130                 target,
131                 existing_username,
132                 existing_pw.value,
133             )
134             self._set_password(service, username, str(password))
135
136     def _set_password(self, target, username, password):
137         credential = dict(
138             Type=win32cred.CRED_TYPE_GENERIC,
139             TargetName=target,

```

```

140         UserName=username,
141         CredentialBlob=password,
142         Comment="Stored using python-keyring",
143         Persist=self.persist,
144     )
145     win32cred.CredWrite(credential, 0)
146
147     def delete_password(self, service, username):
148         compound = self._compound_name(username, service)
149         deleted = False
150         for target in service, compound:
151             existing_pw = self._read_credential(target)
152             if existing_pw and existing_pw['UserName'] == username:
153                 deleted = True
154                 self._delete_password(target)
155         if not deleted:
156             raise PasswordDeleteError(service)
157
158     def _delete_password(self, target):
159         try:
160             win32cred.CredDelete(Type=win32cred.CRED_TYPE_GENERIC, TargetName=target)
161         except pywintypes.error as e:
162             if e.winerror == 1168 and e.funcname == 'CredDelete': # not found
163                 return
164             raise
165
166     def get_credential(self, service, username):
167         res = self._resolve_credential(service, username)
168         return res and SimpleCredential(res['UserName'], res.value)
169

```

15. assistant (2026-06-10T16:24:07.359Z)

The Windows backend raises raw `pywintypes.error` (not `KeyringError`) for transient CredRead failures. Let me verify keyring's core doesn't wrap backend exceptions, and check the errors module hierarchy.

16. user (2026-06-10T16:24:08.877Z)

```

63: def get_password(service_name: str, username: str) -> str | None:
64-     """Get password from the specified service."""
65-     return get_keyring().get_password(service_name, username)
66-
67-
68: def set_password(service_name: str, username: str, password: str) -> None:
69-     """Set password for the user in the specified service."""
70-     get_keyring().set_password(service_name, username, password)
71-
72-
73: def delete_password(service_name: str, username: str) -> None:

```

17. user (2026-06-10T16:24:09.127Z)

```

1     import sys
2     import warnings
3
4
5     class KeyringError(Exception):
6         """Base class for exceptions in keyring"""
7
8
9     class PasswordSetError(KeyringError):
10         """Raised when the password can't be set."""
11

```

```

12
13 class PasswordDeleteError(KeyringError):
14     """Raised when the password can't be deleted."""
15
16
17 class InitError(KeyringError):
18     """Raised when the keyring could not be initialised"""
19
20
21 class KeyringLocked(KeyringError):
22     """Raised when the keyring failed unlocking"""
23
24
25 class NoKeyringError(KeyringError, RuntimeError):
26     """Raised when there is no keyring backend"""
27
28
29 class ExceptionRaisedContext:
30     """
31     An exception-trapping context that indicates whether an exception was
32     raised.
33     """
34
35     def __init__(self, ExpectedException=Exception):
36         warnings.warn(
37             "ExceptionRaisedContext is deprecated; use `jaraco.context.ExceptionTrap`",
38             DeprecationWarning,
39             stacklevel=2,
40         )
41         self.ExpectedException = ExpectedException
42         self.exc_info = None
43
44     def __enter__(self):
45         self.exc_info = object.__new__(ExceptionInfo)
46         return self.exc_info
47
48     def __exit__(self, *exc_info):
49         self.exc_info.__init__(*exc_info)
50         return self.exc_info.type and issubclass(
51             self.exc_info.type, self.ExpectedException
52         )
53
54
55 class ExceptionInfo:
56     def __init__(self, *info):
57         if not info:
58             info = sys.exc_info()
59         self.type, self.value, _ = info
60
61     def __bool__(self):
62         """
63         Return True if an exception occurred
64         """
65         return bool(self.type)
66
67     __nonzero__ = __bool__
68

```

18. user (2026-06-10T16:25:10.659Z)

Structured output provided successfully

19. assistant (2026-06-10T16:25:17.319Z)

Verification complete. The finding's code-structure claims all check out, but the high-severity

trigger mechanism is substantially refuted on the primary platform (Windows), where transient Credential Manager failures raise `pywintypes.error` ? not `KeyringError` ? and therefore crash loudly before any minting, while the swallowed no-backend case also fails the write loudly before any overwrite. Adjusted to medium: a real design gap worth fixing, but the realistic data-loss window is far narrower than claimed.